# Queues

## Operations on queue

- The four main operations on a queue are:
  - Enqueue
  - Dequeue
  - Peek
  - Display

## Implementation using array

### Enqueue

- Enqueue operation inserts an element at the Rear of the queue
- If enqueue is attempted when the queue is full, it results in a Queue *Overflow condition*
- Algorithm

```
Step 1: IF REAR = MAX - 1
            WRITE OVERFLOW
            GOTO Step 4
         [END OF IF]
Step 2: IF FRONT = -1 AND REAR = -1
            SET FRONT = REAR = 0
         ELSE
            SET REAR = REAR + 1
         [END OF IF]
Step 3: SET QUEUE[REAR] = NUM
Step 4: EXIT
```

- Code

```c
void enqueue(int item)
{
  if (rear == MAX-1)
  {
    printf("queue overflow");
    return;
  }
```

```
  if (front == -1 && rear == -1)
    front = rear = 0;
  else
    ++rear;

  queue[rear] = item;
}
```

## Dequeue

- Dequeue operation deletes an element from the Front of the queue and returns it
- If dequeue is attempted when the queue is empty, it results in a *Queue Underflow condition*
- **Algorithm**

```
Step 1: IF FRONT = -1 OR FRONT > REAR
           WRITE UNDERFLOW
           GOTO Step 2
         [END OF IF]
         SET VAL = QUEUE[FRONT]
         SET FRONT = FRONT + 1
Step 2: EXIT
```

- **Code**

```
int dequeue()
{
  int item;

  if (front == -1 || front > rear)
  {
    printf("queue underflow");
    return -1;
  }

  item = queue[front++];
  return item;
}
```

## Peek

- Peek returns the first item of the queue or the item at Front
- Code

```
int peek()
{
  if (front == -1 || front > rear)
  {
    printf("queue empty");
    return -1;
  }

  return queue[front];
}
```

## Display

- Display operation simply traverses the queue and prints every item to the output.
- Code

```
void display()
{
  int index;

  if (front == -1 || front > rear)
  {
    printf("queue empty");
    return;
  }

  for (index = front; index <= rear; ++index)
    printf("%d\t", queue[index]);
}
```

## Full code

```
#include <stdio.h>
#define MAX 10

int queue[MAX];
int front = -1, rear = -1;
```

```c
void enqueue(int);
int dequeue();
int peek();
void display();

int main()
{
  return 0;
}

void enqueue(int item)
{
  if (rear == MAX-1)
  {
    printf("queue overflow");
    return;
  }

  if (front == -1 && rear == -1)
    front = rear = 0;
  else
    ++rear;

  queue[rear] = item;
}

int dequeue()
{
  int item;

  if (front == -1 || front > rear)
  {
    printf("queue underflow");
    return -1;
  }

  item = queue[front++];
  return item;
}

int peek()
{
  if (front == -1 || front > rear)
```

```
  {
    printf("queue empty");
    return -1;
  }

  return queue[front];
}

void display()
{
  int index;

  if (front == -1 || front > rear)
  {
    printf("queue empty");
    return;
  }

  for (index = front; index <= rear; ++index)
    printf("%d\t", queue[index]);
}
```

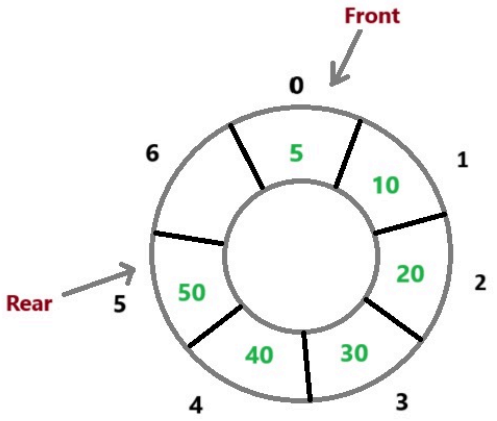
## Types of queue

- A queue data structure can be classified into the following types
  1. Circular queue
  2. Priority queue
  3. Dequeue
  4. Multiple queue

## Circular queue

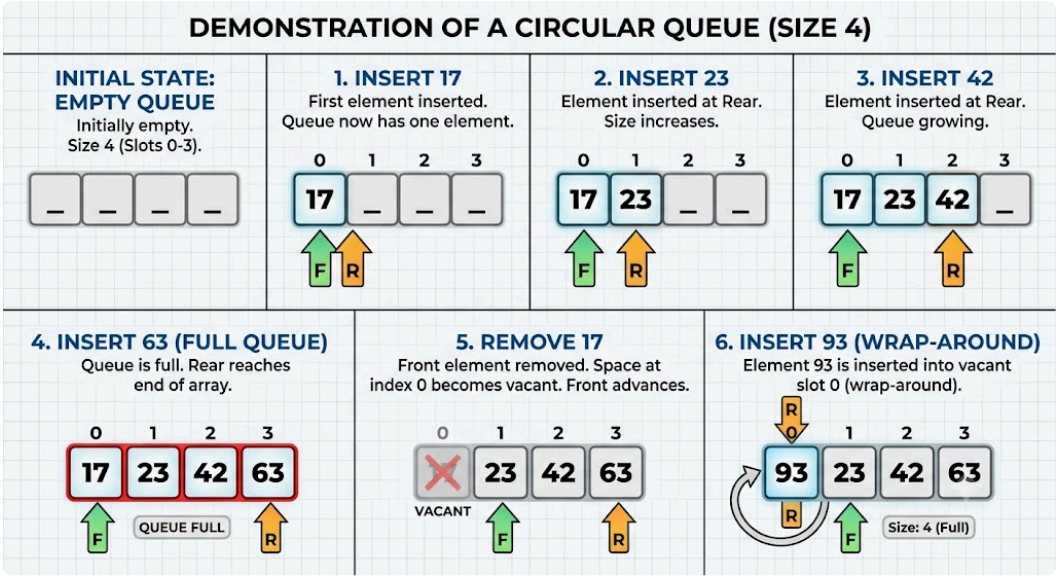
- A circular queue is the extended version of a regular queue where the last element is connected to the first element. Thus

forming a circle-like structure.




## Why is circular queue required?

- The circular queue solves the major limitation of the regular queue. If the regular queue is full and more items are to be inserted, it cannot be done so even if the front of the queue may have empty spaces (items have been deleted before).
- One possible solution is to shift all the items to the left so that vacant spaces can be occupied. But this can be a very time-consuming process, when the queue is quite large.
- As circular queue's Rear wraps around to the start, circular queue can overcome this problem by allowing the insertion of items at the start of the queue when we reach the end of the queue.



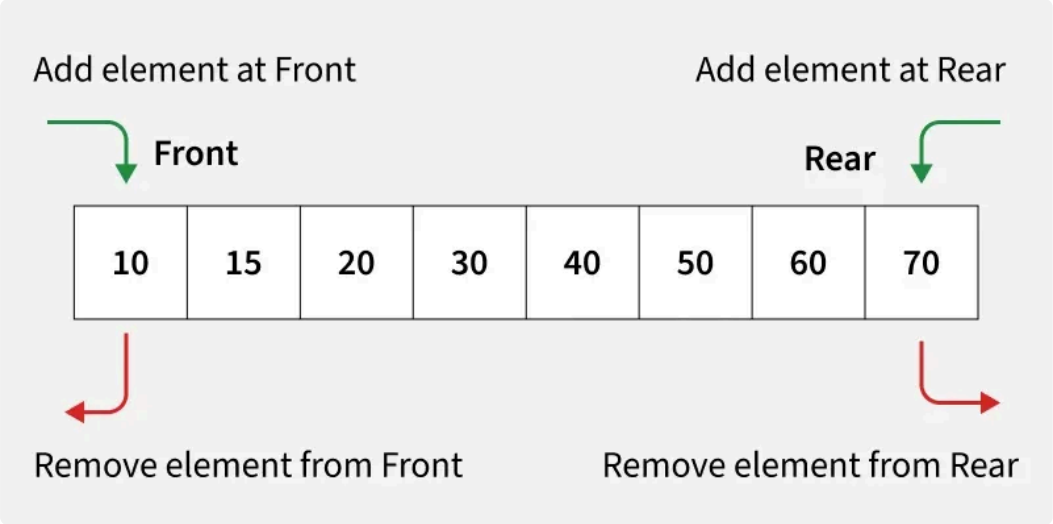

## Priority queue

- A priority queue is a modified queue in which each element is assigned a priority.
- The priority of the element will be used to determine the order in which the elements will be processed.

- The general rule of processing the elements of a priority queue are:
  - An element with higher priority is processed before an element with a lower priority.
  - Two elements with the same priority is processed on a First-Come-First-Serve (FCFS) basis.
- Priority queue are widely used in operating systems to execute the highest priority process first. For example - If there are 3 processes with execution time 5 ns (nanosecond), 4 ns and 7 ns, then the second process will be executed first if the priority is set based on the execution time of processes.

## D-Queue

- D-queue or deque (pronounced as '*deck*' or '*dequeue*') is a special type of queue in which elements can be inserted or deleated at either end.
- In other words, elements can be inserted/deleted at either Front or Rear.




- A dequeue can be implemented using either a circular array or a circular doubly linked list.